Project Report for ECE595: RL Theory and Algorithms
"Counterfactual Data Augmentation for Sample-Efficient RL"

Sajan Kumar
kumar836@purdue.edu

Shilpa Noushad
snoushad@purdue.edu

Pratyush Uppuluri
puppulur@purdue.edu

1. Objective

This project aims to implement and validate a counterfactual-based data augmentation approach, as proposed in the CTRL framework [18], to investigate its effectiveness in addressing two core challenges in practical reinforcement learning — sample efficiency and exploration under limited data regimes. Through empirical studies on benchmark tasks (*Cartpole*, *LunarLander*), the goal is to demonstrate how causal counterfactual reasoning can serve as a viable mechanism for safe and data-efficient policy improvement.

2. Motivation

Reinforcement Learning (RL) has shown remarkable success in simulated settings, but its real-world adoption in domains such as healthcare, robotics, and industrial control remains limited by costly data collection and unsafe exploration. Improving **sample efficiency and exploration** has thus become a central goal in practical RL, motivating approaches that learn effectively from small, fixed datasets while preserving theoretical convergence guarantees.

The CTRL framework (*Sample-Efficient Reinforcement Learning via Counterfactual-Based Data Augmentation*, NeurIPS 2020) introduces a **Structural Causal Model (SCM)** to represent environment dynamics as

$$S_{t+1} = f(S_t, A_t, U_{t+1}), \tag{1}$$

where U_{t+1} denotes exogenous noise. By inferring this latent variable for each observed transition and reusing it to compute **counterfactual next states** $S'_{t+1} = f(S_t, a', U_{t+1})$ for alternate actions, CTRL generates additional, causally consistent experiences without new interactions—enabling **sample-efficient "imaginative exploration"** that broadens data coverage by asking *"what if the agent had taken action a' instead of a?"* The paper shows that if f is monotone in U, counterfactual outcomes are identifiable (Theorem 1 in [18]) and that Q-learning trained on this augmented dataset converges to the optimal value function Q^* (Theorem 2 in [18]), aligning with theoretical guarantees on convergence emphasized in the course. Together, these results make CTRL particularly compelling—uniting causal validity and reinforcement learning optimality within a single, data-efficient framework.

To learn this causal mechanism, CTRL employs a **Bidirectional Conditional GAN (BiCoGAN)[8]** that jointly trains an encoder, generator, and discriminator as in 1, ensuring consistency between causal and inverse mappings, enabling inference of latent noise and reconstruction of next states. With its theoretical grounding and simplicity, CTRL offers a practical, scalable framework to evaluate sample efficiency and exploration on benchmark tasks like *Cartpole* and *LunarLander* [2].

Algorithm 1 Policy Learning via Counterfactual-Based Data Augmentation — Part 1 ([18])

1: **Input:** Observed triplets (S_t, A_t, S_{t+1}) from individuals, for $t = 1, \dots, T$.
2: **Estimation of a general policy:**
3: 2.1. Estimate the SCM given in Eq. 1 using BiCoGAN (as in Eq. 1).
4: 2.2. Generate counterfactual data for alternative actions according to the estimated SCM; denote the counterfactually augmented dataset by $\tilde{\mathcal{D}}$.
5: 2.3. Perform D3QN learning on $\tilde{\mathcal{D}}$ to obtain the learned policy π.

Note: This corresponds to **Part 1** of Algorithm "Pol-

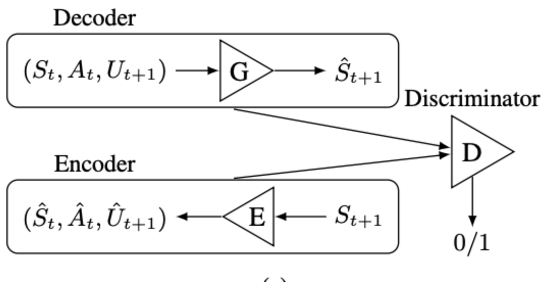

Figure 1. Generator G, Encoder E, and Discriminator D in CTRL [18].

icy Learning via Counterfactual-Based Data Augmentation" in the main framework, following the formulation in [18] which will be implemented and serves framework for experimentation.

3. Background

Reinforcement Learning (RL) formalizes sequential decision-making problems as Markov Decision Processes (MDPs), where an agent interacts with an environment characterized by a state space, action space, transition dynamics, and reward function. The agent's objective is to learn a policy that maximizes expected cumulative reward. In tabular settings or when the environment dynamics are known, classical dynamic programming methods provide exact solutions with strong theoretical guarantees. However, in most practical problems, the state and action spaces are large or continuous, and the transition dynamics are unknown, making exact solutions infeasible.

The advent of deep reinforcement learning (DRL) addressed scalability concerns by combining value-based RL algorithms with function approximation using neural networks. Methods such as Deep Q-Networks (DQN) and their extensions—Double DQN and Dueling DQN—improve stability and reduce estimation bias, enabling RL to scale to high-dimensional problems. Despite these advances, DRL methods remain highly sample-inefficient, often requiring millions of interactions with the environment before converging to a performant policy. This reliance on extensive online exploration limits deployment in real-world domains where data collection is costly, time-consuming, or unsafe.

Offline and batch reinforcement learning aim to mitigate these issues by learning policies from a fixed dataset collected by a behavior policy, without further interaction with the environment. While this paradigm avoids unsafe exploration, it introduces challenges related to limited state–action coverage and distribution shift. In such settings, naïve application of off-policy algorithms can lead to extrapolation error or overly optimistic value estimates. As a result, improving sample efficiency and leveraging limited data more effectively are core challenges in modern RL research.

One promising approach to addressing data scarcity is to augment observed experience using learned models of environment dynamics. Model-based RL methods attempt to learn a transition model and generate synthetic rollouts to improve learning efficiency. However, standard generative models may produce unrealistic transitions, and errors in the learned model can compound during rollouts. These concerns motivate approaches that incorporate structure and causality into the modeling of environment dynamics, ensuring that generated data remains consistent with the true underlying process.

Structural Causal Models (SCMs) provide a principled framework for representing data-generating processes using functional relationships and exogenous noise variables. In the context of RL, SCMs make explicit the causal relationship between states, actions, and next states, enabling counterfactual reasoning. Given an observed transition and inferred latent noise, SCMs allow the computation of counterfactual outcomes corresponding to alternative actions, enabling the generation of additional transitions without new environment interactions. This causal perspective forms the foundation for counterfactual data augmentation methods aimed at improving sample efficiency in offline and data-limited RL settings.

4. Literature Review

While deep reinforcement learning has achieved notable success in data-rich simulated environments, its reliance on large-scale interaction limits applicability in safety-critical and data-scarce domains. These challenges have motivated research directions that seek to improve sample efficiency through better use of existing data, including offline learning, structured generalization, and principled augmentation. Although model-based methods and meta-learning approaches have demonstrated improvements in sample efficiency, their performance often degrades under model bias or distribution shift, motivating alternative approaches grounded in causal reasoning.

Causal inference provides a formal framework for reasoning about interventions and counterfactuals through Structural Causal Models (SCMs) [11]. Foundational work on causal discovery established identifiability conditions under additive noise and post-nonlinear models [6, 13, 20], later extended to more general functional causal models [21]. These results are central to counterfactual reasoning, which requires isolating exogenous noise variables to answer "what-if" queries under hypothetical interventions. Recent work has also examined the limits of counterfactual identifiability in learned neural SCMs, highlighting the importance of structural assumptions such as monotonicity or invertibility [9, 10].

Counterfactual reasoning has been increasingly explored within reinforcement learning to address data inefficiency and off-policy evaluation. Early work introduced counterfactually-guided policy search and data fusion mechanisms to improve policy improvement using observational data [3, 5]. These approaches demonstrated that counterfactual transitions can meaningfully expand the effective support of collected data, but often relied on known or simplified environment models, limiting scalability to complex domains.

The CTRL framework [19] represents a key advance by integrating learned SCMs with deep reinforcement learning. By modeling environment dynamics as a structural causal process and inferring latent exogenous noise via a Bidirectional Conditional GAN [7], CTRL enables generation of causally consistent counterfactual transitions for unobserved actions. Importantly, the framework provides theoretical guarantees: under monotonicity assumptions, counterfactual outcomes are identifiable, and Q-learning on the augmented dataset converges to the optimal value function. This establishes a rare link between causal validity and reinforcement learning optimality.

Subsequent work has expanded and refined counterfactual data augmentation in RL and related settings. Mocoda [12] extends counterfactual augmentation to structured, multi-object domains, while ACAMDA [14] and Rocoda [1] introduce guided and robotics-focused augmentation strategies. Counterfactual reasoning has also been incorporated into offline imitation learning [15] and safe offline RL [16]. Broader surveys on causal reinforcement learning [4, 17] highlight counterfactual augmentation as a promising direction for improving robustness and generalization. Despite this growing body of work, systematic empirical evaluation of counterfactual augmentation under constrained data budgets and standard benchmark environments remains limited, motivating the experimental focus of this project.

5. Methodology

We closely follow the theoretical framework presented in [18] and reproduce their experimental setup as faithfully as possible. Since the authors do not provide an official GitHub repository, and several implementation details in the paper remain ambiguous, we take particular care to avoid unintended deviations. To mitigate bias, we conducted multiple independent experiments to validate that our choices remain aligned with the underlying theory and with the intended behavior described by the authors. Nevertheless, we acknowledge that the lack of publicly available code and the inherent under specification in parts of the original description may lead to unavoidable discrepancies.

In our study, we implement Algorithm 1 from the paper [19] and replicate all major components of CTRL, including the environment dynamics, the SCM assumptions, and the counterfactual augmentation scheme. Our first set of experiments faithfully recreates the CartPole-SD environment used in CTRL as in Stochastic CartPole-SD Environment but performs experiments/evaluations beyond the paper; in subsequent sections, we extend the evaluation to `LunarLander-v3` to demonstrate how the methodology generalizes beyond the original setting and to explore its behavior in more complex domains and finally we evaluate as comparison to regular CartPole environment at different mixing proportions.

5.1. Stochastic CartPole-SD Environment (Our Modifications)

We follow the SD setting from [19] but explicitly document the modifications introduced in our implementation. The underlying CartPole dynamics remain unchanged; only the action parameterization, noise model, and termination logic differ. The agent selects among 11 discrete actions $a \in \{0, \dots, 10\}$, which are mapped to a continuous control value $a_{\text{cont}} = a/10$ and further perturbed by Gaussian execution noise $\tilde{a} = a_{\text{cont}} + \epsilon$, with $\epsilon \sim \mathcal{N}(0, 0.05^2)$. The resulting applied force is $F = (2\tilde{a} - 1) F_{\max}$ with $F_{\max} = 10$. Although the original CTRL work does not clearly specify its termination implementation, we adopt a choice that is consistent with the environment physics: termination is evaluated on the *clean* next state prior to adding any stochastic noise, i.e., whenever $|x'_{\text{clean}}| > 2.4$ or $|\theta'_{\text{clean}}| > 12°$. After termination is checked, the environment returns a noisy observation $s' = s'_{\text{clean}} + 0.05\,\eta$, with $\eta \sim \mathcal{N}(0, I_4)$, subsequently clipped to the usual CartPole bounds $x' \in [-4.8, 4.8]$ and $\theta' \in [-0.418, 0.418]$. Dataset generation follows the original SD protocol: each episode samples random discrete actions, applies the noisy action $\tilde{a}$, and records $(s, a, \tilde{a}, r, s')$ until termination.

We generate the SD dataset exactly as in the original CTRL setting: each episode samples random discrete actions, applies the noisy action $\tilde{a}$, and records $(s, a, \tilde{a}, r, s')$ until termination.

.

5.2. Offline D3QN + CQL Training with Real and Counterfactual Data

After training the BiCoGAN as in 2, we construct the offline RL dataset by combining real SD transitions with counterfactual (CF) transitions generated via the learned (G, E) pair. Given a real transition (s_t, a_t, r_t, s_{t+1}), we generate k counterfactual transitions by sampling latent noise u and replacing the original action with all alternative actions as in the original CTRL procedure:

$$\hat{s}' = G(s_t, a_{\text{cf}}, u), \qquad a_{\text{cf}} \neq a_t.$$

Each CF transition inherits the reward r_t and termination flag based on the clean termination thresholds applied to $\hat{s}'$.

We evaluate two regimes:

- **Real-only**: the offline dataset consists solely of the clean SD transitions.
- **Real + CF (k per real)**: each real transition is augmented with k counterfactual transitions generated from BiCoGAN.

Algorithm 2 BiCoGAN Training Pipeline (Our Implementation)

Require: Dataset $\mathcal{D} = \{(s_t, a_t, \tilde{a}_t, r_t, s_{t+1})\}$, hyperparameters $(\alpha, \rho, \phi, \lambda_{\text{fwd}})$.
Ensure: Trained (G, E, D) models.

```
Stage 1: Pretrain Generator G
for epoch = 1 to N_pre do
    for minibatch (s_t, a_t, ã_t, s_{t+1}) from D do
        Predict ŝ_{t+1} = G(s_t, a_t, u = 0).
        Update G via MSE(ŝ_{t+1}, s_{t+1}).
    end for
end for
Stage 2: Adversarial BiCoGAN Training
for epoch = 1 to N_GAN do
    for minibatch (s_t, a_t, ã_t, s_{t+1}) do
        Sample latent u ~ N(0, I).
        Generate fake ŝ_{t+1} = G(s_t, a_t, u).
        Discriminator update:
```

$$\mathcal{L}_D = \text{BCE}(D(s_t, a_t, s_{t+1}), 1) + \text{BCE}(D(s_t, a_t, \hat{s}_{t+1}), 0)$$

```
        Gradient step on D.
        Encoder–Generator update:
        (ŝ_t, â_t, û_t) = E(s_{t+1}).
        ŝ'_{t+1} = G(s_t, a_t, û_t).
        Compute
```

$$\mathcal{L}_{GE} = \mathcal{L}_{\text{adv}} + \gamma_t \mathcal{L}_{\text{EFL}} + \lambda_{\text{fwd}} \mathcal{L}_{\text{fwd}}.$$

```
        Gradient step jointly on G and E.
    end for
end for
```

All states and next states are standardized using the real dataset mean and standard deviation. We train an offline Dueling D3QN agent as in 3 with a Conservative Q-Learning (CQL) penalty, following the update structure of [19]. The value network is trained using Double DQN targets and soft target updates.

Reproducibility. All experiments, result plots, and evaluation logs used in this section can be fully reproduced using our publicly released notebook available at `Counterfactual-RL Experiments`.

Algorithm 3 Offline D3QN + CQL Training (Real or Real+CF Data)

Require: Normalized dataset (S, A, R, S', D), hyperparameters γ, α_{CQL}, τ.
Ensure: Trained Q-network.

Initialize Q-network Q_θ and target network Q_{θ^-}.
for epoch $= 1$ to N_{epochs} **do**
  Shuffle dataset.
  for each minibatch (s, a, r, s', d) **do**
    Compute $Q_\theta(s, a)$ and $Q_\theta(s, \cdot)$.
    Compute Double DQN target:

$$y = r + \gamma Q_{\theta^-}\left(s', \arg\max_{a'} Q_\theta(s', a')\right)(1 - d).$$

    TD loss:

$$\mathcal{L}_{\text{TD}} = \text{Huber}(Q_\theta(s, a), y).$$

    CQL penalty:

$$\mathcal{L}_{\text{CQL}} = \alpha_{\text{CQL}} \left(\log \sum_{a'} e^{Q_\theta(s,a')} - Q_\theta(s, a) \right).$$

    Total loss:

$$\mathcal{L} = \mathcal{L}_{\text{TD}} + \mathcal{L}_{\text{CQL}}.$$

    Gradient update on Q_θ.
    Soft target update:

$$\theta^- \leftarrow \tau\, \theta + (1 - \tau)\theta^-.$$

  end for
  if epoch mod eval_every $= 0$ **then**
    Evaluate policy for 50 rollouts in the clean CTRL environment.
  end if
end for

5.3. Experiments on LunarLander-v3

For our second environment, we use the standard `LunarLander-v3` from Gymnasium, with default physics (gravity, wind, and turbulence parameters), and no modifications to the underlying dynamics.

BiCoGAN SCM for LunarLander. We reuse the same Structural Causal Model (SCM) framework developed for CartPole-SD: a bidirectional conditional GAN that learns forward dynamics $G(s_t, a_t, u_t)$ and an encoder $E(s_{t+1})$ for recovering latent exogenous structure. The architecture, loss functions, and training schedule are identical to the CartPole version; only the state and action dimensions differ.

Base-S World Model. To compare SCM-based counterfactual generation with a simpler probabilistic world model, we implement *Base-S*, a Gaussian next-state-and-reward model following the formulation in the CTRL paper. Base-S conditions on the current state and a one-hot action vector and produces

$$(\mu_{s'}, \mu_r, \log \sigma_{s'}^2, \log \sigma_r^2),$$

the means and log-variances of a diagonal Gaussian distribution over next state and reward. Samples are drawn via reparameterization, and the model is trained using a Gaussian negative log-likelihood loss:

$$\mathcal{L}_{\text{NLL}} = \mathbb{E}\left[\mathcal{N}(s_{t+1}; \mu_{s'}, \sigma_{s'}^2) + \mathcal{N}(r_t; \mu_r, \sigma_r^2)\right].$$

Training Base-S. We train Base-S for 20 epochs using Adam with learning rate 10^{-3}, weight-normalized MLP layers, and minibatch NLL over both next state and reward. This follows the training setup stated in the CTRL paper and provides a baseline generative model for producing counterfactual transitions. The trained Base-S model is then used to generate synthetic rollouts for offline RL, enabling a direct comparison with the BiCoGAN SCM approach.

6. Outcomes and Discussion

6.1. BiCoGAN Training Dynamics

Figure 2 shows the training curves for the discriminator loss, generator adversarial loss, encoder consistency loss, and the $\gamma(t)$ schedule used to weight the encoder feature loss (EFL). All plots are averaged per epoch over the full SD dataset.

During training, the discriminator quickly enters a stable regime, indicating that real and generated transitions become difficult to separate after pretraining. The generator adversarial loss decreases smoothly as

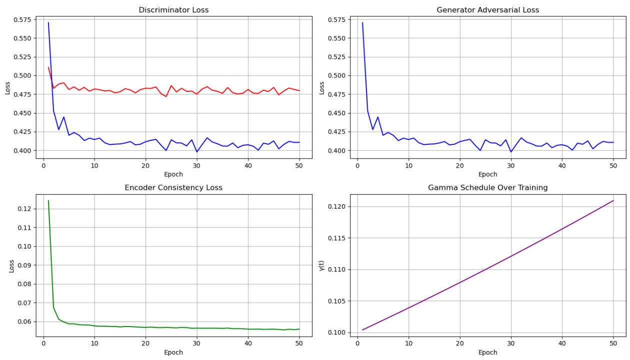

Figure 2. BiCoGAN training curves. **Top-left:** discriminator loss for real and fake samples converges toward the stable range ≈ 0.40–0.48. **Top-right:** generator adversarial loss steadily decreases and stabilizes after $\sim$10 epochs. **Bottom-left:** encoder consistency loss (EFL) rapidly decays during early training and then plateaus, indicating stable backward reconstruction. **Bottom-right:** the $\gamma(t)$ schedule increases exponentially per the CTRL formulation, gradually strengthening the contribution of encoder consistency.

G adapts to match the transition distribution. The encoder consistency loss drops sharply in the first few epochs, consistent with E learning the inverse mapping from s_{t+1} to (s_t, a_t, u_t). Finally, the increasing $\gamma(t)$ schedule progressively emphasizes semantic consistency, as in the original CTRL design [18].

6.2. Offline D3QN+CQL Results: Real vs. Counterfactual Training

We compare the performance of an offline Dueling D3QN + CQL agent trained on (a) real SD transitions only, and (b) real transitions augmented with k counterfactual (CF) transitions generated by our BiCoGAN model. All experiments use the CTRL-style SD environment described earlier, and we match the implementation details of [18] as faithfully as possible. However, as discussed below, our empirical gains do not reach the magnitude reported in the original paper, despite reproducing the critical components of the setup.

Distribution of evaluation returns. Figure 3 shows the distribution of mean evaluation returns across 50 evaluation checkpoints. Real-only training produces a tight cluster of returns around 20–30, whereas CF-augmented training yields a substantially wider distribution with a heavier right tail, occasionally reaching mean returns above 40–50. This indicates that CF augmentation can produce higher-performing policies, but also introduces significant variance in training outcomes.

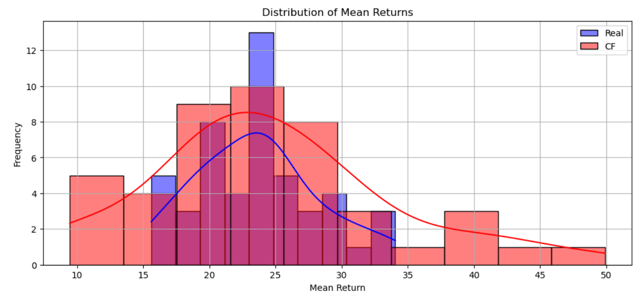

Figure 3. Distribution of evaluation mean returns for Real-only vs. Real+CF. CF-augmented policies occasionally attain substantially higher returns, but also exhibit increased variability compared to Real-only.

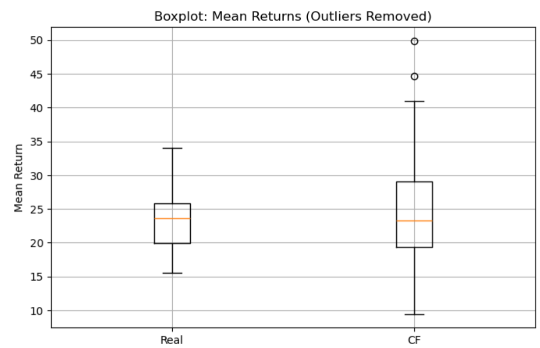

Figure 4. Boxplot of per-seed mean returns (outliers removed). CF augmentation increases the spread of returns and produces high-performing outliers, but the median remains similar to the Real-only baseline.

Per-seed statistics. Figure 4 presents a boxplot of per-seed mean returns with outliers removed. The median performance of Real+CF remains close to the Real-only median, but the interquartile range is larger and the CF setting displays several high-performing outliers (40–50 mean return). This suggests that counterfactual transitions can improve the upper bound of achievable policy quality but do not guarantee consistent improvements across all seeds.

Performance over training. Figure 5 shows the evaluation curve over offline training epochs. CF-

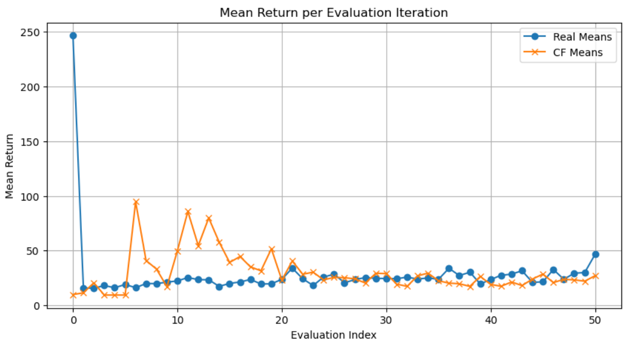

Figure 5. Evolution of mean evaluation return during training. Real-only agents remain stable in the 20–30 range. CF-augmented agents show early high-return spikes but also exhibit increased variability and do not consistently surpass the Real-only baseline over the full training horizon.

augmented agents exhibit early spikes in performance (60–100 return range) that Real-only agents never achieve, reflecting improved sample efficiency for certain seeds. However, the CF performance later stabilizes near Real-only levels and displays more fluctuation, consistent with greater sensitivity to model error in the BiCoGAN-generated counterfactual transitions.

6.3. Hybrid Counterfactual Training on LunarLander-v3

We evaluate Hybrid CTRL training on `LunarLander-v3`, where a D3QN agent is trained either on real transitions only, or on a hybrid mixture of real transitions and counterfactual (CF) samples generated by Base-S or by our BiCoGAN SCM.

Setup. In this setup, we use counterfactual samples generated by both Base-S and BiCoGAN from previously stored transitions, and train the D3QN agent using a 50% REAL + 50% CF mixing strategy during offline updates. All agents share identical hyperparameters; only the source of CF data differs.

Comment. Counterfactual augmentation (Table 1) increased the variability of learned policies on `LunarLander-v3`, with neither Base-S nor BiCoGAN providing consistent improvements over the Real-only agent. However, BiCoGAN CF achieved slightly better mean performance than Base-S and produced occasional higher-return trajectories, despite overall instability. .

Table 1. Evaluation returns on real `LunarLander-v3` after 50 training episodes (mean $\pm$ std over 10 rollout episodes).

Agent	CF Source	Mean Return $\pm$ Std
Real-only	None	-29.95 ± 47.84
Hybrid CTRL	Base-S	-95.68 ± 150.48
Hybrid CTRL	BiCoGAN	-23.50 ± 101.16

6.4. Setup 2: Counterfactual Selection via Bellman Score

In this setup, we improve the Hybrid CTRL procedure by selecting only the *best* counterfactual transition for each real transition (s_t, a_t, r_t, s_{t+1}). For every alternative action $a_{\text{cf}} \neq a_t$, we generate a counterfactual next-state using either Base-S or BiCoGAN and evaluate it using a one-step Bellman score.

Bellman-score selection logic. For each candidate counterfactual action a_{cf}:

1. The SCM (Base-S or BiCoGAN) predicts:

$$\hat{s}'_i, \quad \hat{r}_i, \quad \hat{d}_i \text{ (BiCoGAN only).}$$

2. We compute a Bellman score:

$$B_i \;=\; \hat{r}_i \;+\; \gamma(1-\hat{d}_i)\,\max_{a'} Q_{\text{target}}(\hat{s}'_i, a').$$

3. The counterfactual with the highest score is stored:

$$(a_{\text{best}},\; \hat{s}'_{\text{best}},\; B_{\text{best}}).$$

Stored transition. Each selected counterfactual entry contains:

$$(s_t,\; a_{\text{best}},\; \hat{r}_{\text{best}},\; \hat{s}'_{\text{best}},\; \hat{d}_{\text{best}}).$$

Evaluation. We train a D3QN agent with 20 evaluation rollouts per setting and report the mean and standard deviation over the rollouts.

Table 2. Setup 2: Final evaluation returns (20 episodes) using Bellman-score-selected counterfactuals.

Agent	**CF Source**	**Mean Return ± Std**
Real Only	None	-145.95 ± 55.89
Hybrid CTRL (best-CF)	Base-S	-326.99 ± 116.05
Hybrid CTRL (best-CF)	BiCoGAN	-262.48 ± 86.85

Comment. Even with Bellman-score selection 2, counterfactual augmentation did not yield performance improvements on `LunarLander-v3`; instead, both Base-S and BiCoGAN produced highly variable and generally lower-performing policies compared to Real-only training,lunar lander experiments can be observed and run with code at `Counterfactual-RL LunarLander Experiments`.

Our contributions. In addition to faithfully reproducing the CTRL experimental setting, we extend the original CartPole-SD analysis by introducing new offline evaluations that systematically vary the proportion of real and counterfactual data. These experiments compare Real-only and Real+CF training under Offline D3QN+CQL and analyze performance using distributional statistics histogram: 3, boxplot:4, providing a finer-grained study than reported in the original work.

Separately, we introduce LunarLander-v3 as a new evaluation environment. For LunarLander, we study two counterfactual training regimes: (i) a fully offline setting, where counterfactual transitions are pre-generated and mixed with real data using a fixed 50% real and 50% counterfactual ratio during training, and (ii) a hybrid setting, where the agent interacts with the environment online and generates counterfactual samples on-the-fly using a Bellman-score-based selection criterion results reported in Table 1 and Table 2 respectively . In both cases, we compare counterfactual samples generated by Base-S and BiCoGAN SCMs, Finally, we present a quantitative comparison of different mixing proportions of real and counterfactual data on both the noisy and clean CartPole environments, as reported in Table 3.

7. Results

We consolidate our CartPole-SD reproduction runs (offline) using three agents: Rainbow, D3QN trained on real data only, and D3QN trained on mixed real + counterfactual (CF) data from the BiCoGAN SCM. Each experiment reports the mean return over 50 evaluation rollouts on the clean (non-noisy) environment; CTRL-style noisy evaluation is also reported for D3QN settings to mirror the original paper.

Table 3. CartPole-SD offline evaluation (mean return ± std over 50 rollouts).

Agent	CF frac	Clean	CTRL (noisy)
Rainbow	–	10.9 ± 1.7	N/A
D3QN (real only)	0	34.4 ± 22.3	16.7 ± 9.2
D3QN + CF	0.25	52.4 ± 25.6	18.6 ± 10.7
D3QN + CF	0.10	51.3 ± 26.6	19.1 ± 12.6
D3QN + CF	0.05	127.9 ± 66.1	21.1 ± 12.9

Across settings, counterfactual augmentation consistently lifts performance on the clean environment while CTRL-style noisy evaluation remains clustered near 18–21 mean return. The largest gains appear when the CF sampling fraction is small (0.05), suggesting moderate augmentation helps without overwhelming the replay buffer with model-generated transitions.

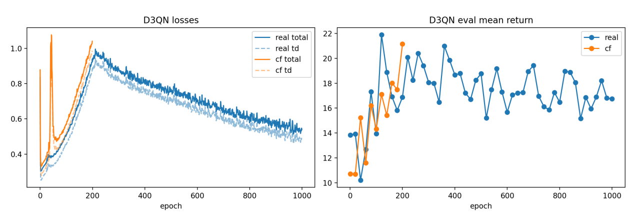

Figure 6. D3QN training curves showing losses and evaluation returns for real vs. CF training, highlighting stability on CTRL (noisy) evaluation and larger gains on clean evaluation.

8. Conclusion and Future Work

Counterfactual augmentation using modelled structural causal models (SCMs) is a powerful theoretical technique: it allows agents to explore alternative transitions without interacting with the real environment, and in principle enables safer, broader exploration.

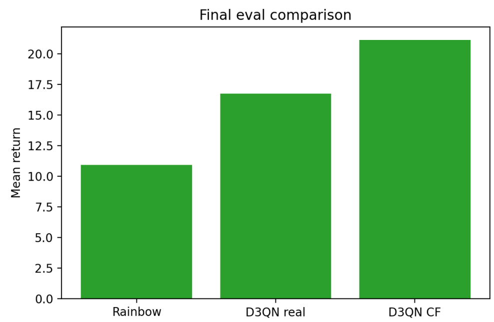

Figure 7. Final mean-return comparison (clean): bar plot over Rainbow, D3QN real-only, and the three CF fractions.

However, our empirical results show that counterfactual augmentation can be highly variable in practice. Although counterfactual samples occasionally help the agent discover higher–return trajectories, both Base-S and BiCoGAN SCMs also introduce substantial noise and instability during offline training.

Our implementation closely follows the theoretical framework proposed in the CTRL paper, in which counterfactual outcomes are defined through the following SCM:

$$S_{t+1} = f(S_t, A_t, U_{t+1}), \qquad U_{t+1} \perp\!\!\!\perp (S_t, A_t),$$

where f is assumed to be smooth and strictly monotonic in the exogenous noise variable U_{t+1} for fixed (S_t, A_t). Under these conditions, Theorem 1 of the CTRL paper guarantees that the counterfactual outcome

$$S_{t+1,\, A_t=a'} \;\mid\; (S_t = s_t,\ A_t = a,\ S_{t+1} = s_{t+1})$$

is identifiable. The GAN-based generator used in the original CTRL paper leverages this monotonicity assumption implicitly; however, in practice the adversarial training dynamics and limited model capacity may violate the smoothness or monotonicity conditions required for clean identifiability. Exploring monotonic neural architectures SCMs with explicit invertibility may improve the reliability of counterfactual samples.

A promising future direction is to evaluate whether counterfactual augmentation improves *generalization* across environment parameter shifts. For instance, in `LunarLander-v3` one could systematically vary gravity, wind turbulence, or initial horizontal velocity, and test whether CF-augmented agents adapt better to such perturbations than real-only agents.

Finally, our study used D3QN as the base offline RL algorithm, mirroring the original CTRL work. An important extension would be to test counterfactual augmentation across a broader set of RL methods, including value-based (Q-learning variants), policy-gradient (e.g., PPO, TRPO), and trust-region or constrained offline RL methods. Such comparisons would clarify whether the observed variability is due to the SCM quality, the Bellman objective, or the interaction between the two.

Overall, while counterfactual augmentation remains theoretically appealing, our experiments highlight the importance of model identifiability, counterfactual quality, and algorithmic robustness when deploying SCM-based methods in practical reinforcement learning settings.

Reproducible implementation. All experiments described above can be fully reproduced using our public notebook and code is available `ECE595-Causal-RL-Study`.

References

[1] Edoardo Ameperosa, Jack A. Collins, and Mayank Jain. Rocoda: Counterfactual data augmentation for data-efficient robot learning from demonstrations. In *Proceedings of the IEEE International Conference on Robotics and Automation*, 2025. 3

[2] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym. `https://arxiv.org/abs/1606.01540`, 2016. arXiv:1606.01540. 1

[3] Lars Buesing, Theophane Weber, Yori Zwols, Sebastien Racaniere, Arthur Guez, and Nicolas Heess. Woulda, coulda, shoulda: Counterfactually-guided policy search. In *International Conference on Learning Representations (ICLR)*, 2019. 3

[4] Zhiwei Deng, Jing Jiang, Guodong Long, and Chengqi Zhang. Causal reinforcement learning: A survey. *arXiv preprint arXiv:2307.01452*, 2023. 3

[5] Andrew Forney, Judea Pearl, and Elias Bareinboim. Counterfactual data-fusion for online reinforcement learners. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1156–1164, 2017. 3

[6] Patrik O. Hoyer, Dominik Janzing, Joris M. Mooij, Jonas Peters, and Bernhard Schölkopf. Nonlinear causal discovery with additive noise models. In *Ad-*

vances in Neural Information Processing Systems, 2009. 3

[7] Ayush Jaiswal, Wael AbdAlmageed, Yue Wu, and Premkumar Natarajan. Bidirectional conditional generative adversarial networks. *arXiv preprint arXiv:1711.07461*, 2017. 3

[8] Ayush Jaiswal, Wael AbdAlmageed, Yue Wu, and Premkumar Natarajan. Bidirectional conditional generative adversarial networks. In *Computer Vision – ACCV 2018*, pages 216–232, Cham, 2019. Springer International Publishing. 1

[9] Alireza Nasr-Esfahany and Emre Kiciman. Counterfactual (non-)identifiability of learned structural causal models. *arXiv preprint arXiv:2301.09031*, 2023. 3

[10] Michael Oberst and David Sontag. Counterfactual off-policy evaluation with gumbel-max structural causal models. *arXiv preprint arXiv:1905.05824*, 2019. 3

[11] Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, Cambridge, UK, 2000. 3

[12] Silviu Pitis, Elliot Creager, Ajay Mandlekar, Frederik Ebert, Sebastian Tschiatschek, Chelsea Finn, and Sergey Levine. Mocoda: Model-based counterfactual data augmentation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. 3

[13] Shohei Shimizu, Patrik O. Hoyer, Aapo Hyvärinen, and Antti J. Kerminen. A linear non-gaussian acyclic model for causal discovery. *Journal of Machine Learning Research*, 7:2003–2030, 2006. 3

[14] Yifan Sun, Enbo Wang, Biwei Huang, Changjie Lu, and Li Feng. Acamda: Improving data efficiency in reinforcement learning through guided counterfactual data augmentation. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2024. 3

[15] Zhen Sun, Bowen He, Jun Liu, Xi Chen, and Chao Ma. Offline imitation learning with variational counterfactual reasoning. *Advances in Neural Information Processing Systems*, 2023. 3

[16] Yuan Yao, Zhe Cen, Wenlong Ding, Hanyu Lin, and Shuai Liu. Oasis: Conditional distribution shaping for offline safe reinforcement learning. *Advances in Neural Information Processing Systems*, 2024. 3

[17] Yang Zeng, Rui Cai, Feng Sun, and Longbo Huang. A survey on causal reinforcement learning. *IEEE Transactions on Neural Networks and Learning Systems*, 2024. 3

[18] Jun Zhang, Dragomir Radev, Le Song, Devendra Subramanian, Haoran Xu, Wenlong Liao, and Jimeng Sun. Sample-efficient reinforcement learning via counterfactual-based data augmentation. In *Proceedings of the Neural Information Processing Systems (NeurIPS) Workshop on Causal Discovery and Causality-Inspired Machine Learning*, 2020. arXiv:2012.09092. 1, 2, 3, 6

[19] Jun Zhang, Dragomir Radev, Le Song, Devendra Subramanian, Haoran Xu, Wenlong Liao, and Jimeng Sun. Sample-efficient reinforcement learning via counterfactual-based data augmentation. *NeurIPS Workshop on Causal Discovery and Causality-Inspired Machine Learning*, 2020. arXiv:2012.09092. 3, 4

[20] Kun Zhang and Aapo Hyvärinen. On the identifiability of the post-nonlinear causal model. In *Proceedings of the Conference on Uncertainty in Artificial Intelligence*, 2009. 3

[21] Kun Zhang, Zhikun Wang, Jun Zhang, and Bernhard Schölkopf. On estimation of functional causal models: General results and application to the post-nonlinear causal model. *ACM Transactions on Intelligent Systems and Technology*, 7(2):13, 2016. 3